

On this page

Fields

text	Text field that lets you introduce any free text.	• multi (Boolean) - Whether the input can have multiple values. • trim (Boolean) - Configures a search component when all text fields need to be trimmed.
number	Numeric type field.	N/A
select	Select box with fixed options.	• multi (Boolean) - Whether the select can have multiple selected values. • options (List<Object>) - List of options to be listed in the select. Each object must have a <i>label</i> and <i>value</i> property. • renderType (String) - Allows you to define the text that appears in both the selected values and the options. Can be one of: "label", "value" or "label-value". • sort (Boolean) - Allows select options to be ordered by alphanumerics. ◦ Default value: true
async-select	Asynchronous select box.	• multi (Boolean) - Whether the select can have multiple selected values. • resource (String) - Resource from where the options should be fetched. • renderType (String) - Allows you to define the text that appears in both the selected values and the options. Can be one of: "label", "value", "label-value" or "template". In case the renderType is "template", you can check the property template under this page available to build the template string. • labelKey (String) - The async select expects the options to have a <i>label</i> and <i>value</i> properties defined. When the resource response does not follow this format, it is necessary to map those properties to the ones that are expected by the select box. The <i>labelKey</i> property refers to the property of the fetched object that corresponds to the label that should appear in the select options. • valueKey (String) - Same as the above, but this time it corresponds to the select value. • groupKey (string) - To have option groups, we need to select what is the key to group the elements. Since this is an async select, this

		group key also needs to be on the <i>orderBy</i>, so the BE sends the elements ordered. • groupType (string) - What is the data type that gets displayed as group title. • orderBy (Array<String>) - order of the select elements. • optionsFilter (string) - Allows to filter the select options based on the resource endpoint. For example, if resource is set to "queue", by using <i>optionsFilter: "'type' eq 'EVENT'"</i>, we filter the queues of type "EVENT" from the "queue" API endpoint. • extraOptions (Array) - allows to add any value to these selection lists. Configuring this property allows users to, for example, search for unassigned alerts using like this: <pre> "extraOptions": [{ "label": "placeholder.assigned.unassigned", "value": "null" }, { "label": "placeholder.assigned.assigned", "value": "nonnull" }] </pre> • sort (Boolean) - Allows select options to be ordered by alphanumerics. ◦ Default value: true
datepicker	Datetime picker.	N/A
range	Range of fields.	• fields (Array<Object>) - List of fields that belong to the range. Each object has the same properties as a normal field. • maxRange (Object Number) - The max range property works when you want to limit the range of the inputs; Currently we support two range inputs, number and datepickers; An example of the configuration for a datepicker is: <pre> { value: 365 unit: "days" //where the unit is one that is defined in https://momentjs.com/docs/#/durations/ library } </pre> For number inputs we simply specify a number.
eventStatusReasons		

	Configures a select field that allows to filter by status reasons of a specific state machine. This field can only be configured in a search form that has a resource of type event. In an Omnichannel environment we can specify the section where we want the select to be displayed by adding a key property like {channelId}#.	• stateMachine (string) - Holds the state machine unique identifier. From the available global properties, this type only needs the multi and cssClass property.
eventStatus	Configures a select field that allows to filter by status of a specific state machine. This field can only be configured in a search form that has a resource of type event. In an Omnichannel environment we can specify the section where we want the select to be displayed by adding a key property like {channelId}#.	• stateMachine (string) - Holds the state machine unique identifier. From the available global properties, this type only needs the multi and cssClass property.
accessGroup	Configures an select filter that will enable the user to filter by access group.	The only configuration that is required is setting the property type as <code>accessGroup</code> . If we want to extend the base configuration we can do it by passing extra properties. The available properties are specified in the <code>select</code> type (in this table).
template	Configures a Template for tokenized fields, see Tokenization Configuration to learn more.	• key (string) - The key property, is a property that holds the name of the field that has the information to unlock the tokenized field. • reveal (string) - The reveal property is a property that contains the name of the field that will be shown after unlocking. • templateString (string) - The templateString property is a property where it is put as the field will be shown before unlocking, commonly used in the form of asterisks, something like "*****".
rules	Configures a select field that allows to filter by rule name within a specific rules project. In an Omnichannel environment it will retrieve the available rules from all Pulse applications workflows.	The only configuration that is required is setting the property type as <code>rules</code> . To extend the base configuration, consider the following properties: • title (string) - The title of the field. Default value is <code>event.rules.name</code> . • cssClass (string) - The CSS class that holds custom styles for the field. • key (string) - The key defined for the field. Default value is <code>rules.id</code> . • filter (string) - The filter operator defined for the field. Default value is <code>in</code> . • multi (boolean) - Whether the select can have multiple selected values. Default value is <code>true</code> .

		• clearable (boolean) - Whether the field values can be cleared. Default value is <code>true</code>. • placeholder (string) - The placeholder for the field. Default value is <code>form.placeholder.select</code>.
--	--	---

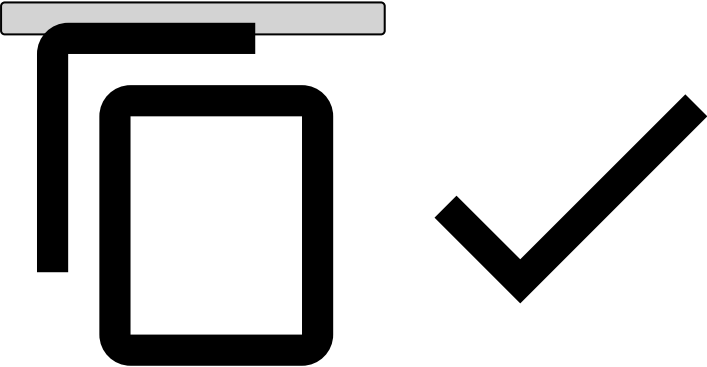
Configuring the select options values

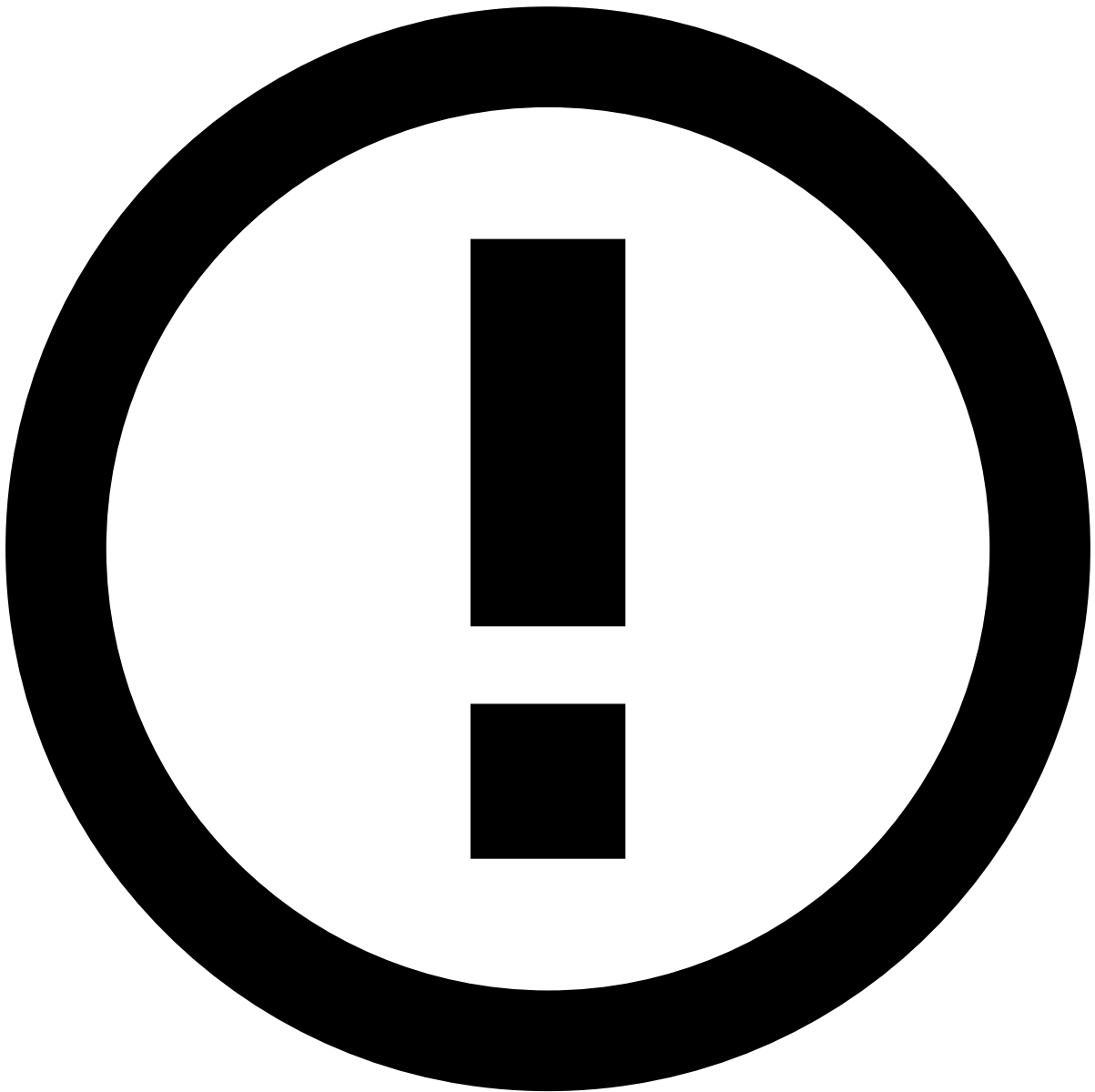
You can set a configuration for a value that needs to be calculated. If this is the case, the value property must be an object:

type	Yes	String	Specifies the type of the value to be calculated. Currently we support the following types: • userId - The value of the option will be the session.id of the user. • subtractTime - The value of the option will be a timestamp value, corresponding to an amount of time subtracted to the current date.
value	Yes, if the type is subtractTime	Number	Amount of the unit time that you want to subtract.
unit	Yes, if the type is subtractTime	String	Unit of the time. Supported: "day", "week", "month", "year"

For example, if you want that the option has a value of 5 years ago, you can set:

```
{
  label: "label.five.years",
  value: {
    type: "subtractTime",
    value: 5,
    unit: "year"
  }
}
```





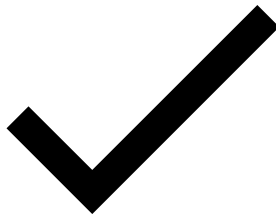
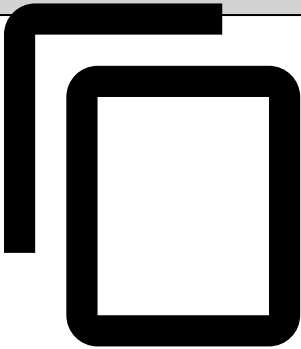
info

The result value will always be the beginning of the result day that you get after the subtract operation (hours/minutes/seconds are ignored for this purpose). Examples:

08/03/2018 4pm	1 day	07/03/2018 12am
08/03/2018 4pm	5 year	08/03/2013 12am
08/03/2018 4pm	0 day	08/03/2018 12am
08/03/2018 4pm	0 month	08/03/2018 12am
08/03/2018 4pm	1 month	08/02/2018 12am
08/03/2018 4pm	1 hour	08/03/2018 12am
08/03/2018 4pm	-1 day	09/03/2018 12am

You can also not define any value. This means that the filter is ignored if the user selects that option. For instance:

```
{  
  label: "label.all.time"  
}
```



Configuring the select default value📄



Attention

Default values aren't available for selects of type **async-select**.

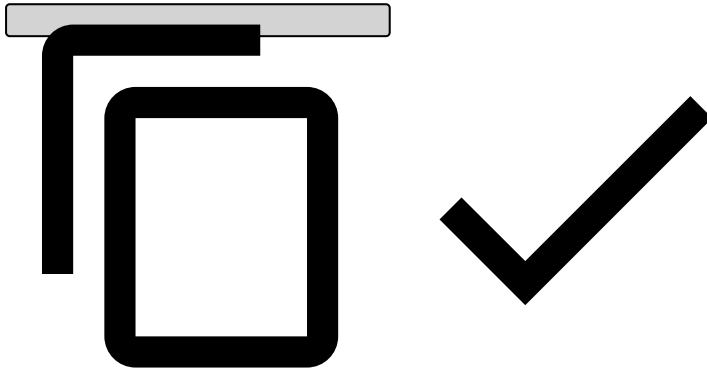
The default value is an object with a property label, corresponding to the label of the option that must be set as default. For instance:

```
{  
  options: [  
    {  
      label: 'Option 1',  
      value: 1,  
    },  
    {  
      label: 'Option 2',  
      value: 2,  
    },  
  ],  
}
```

```

{
  label: "a",
  value: 1
},
{
  label: "b",
  value: 2
}
]
defaultValue: {
  label: "a"
}
}

```



This configuration will set the option with the label "a" as default.

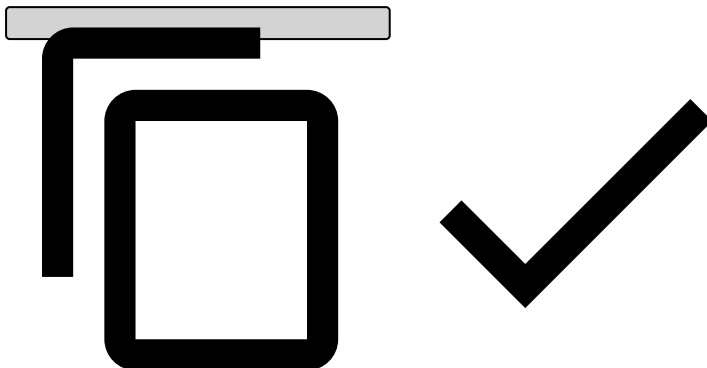
Configuring the default value of a datepicker [🔗](#)

You can set a default value for the datepicker field. To do that, you must configure an object, inside of the datepicker object, like this one:

```

{
  "type": "datepicker",
  ...
  "defaultVal": {
    "type": "subtractTime",
    "value": 5,
    "unit": "year"
  }
}

```

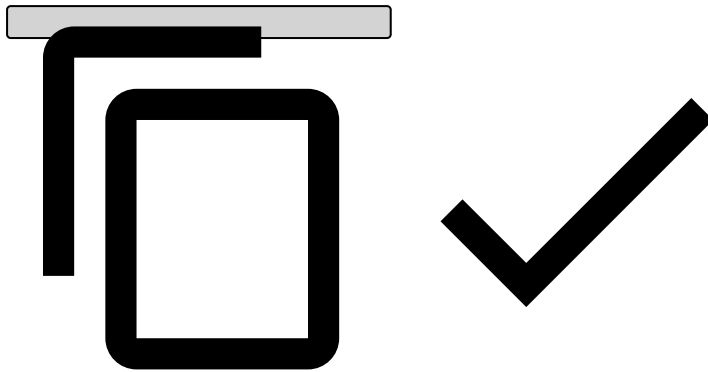


Please read the above documentation for configuring the select options to find more about how to configure this value. Please note that, for the datepicker, you can only use the "subtractTime" type.

Configuring the default value of other fields [🔗](#)

Besides select and datepicker fields, all fields accept a default value. To set that value you must configure the field object, adding the **"defaultVal"** property as shown:

```
{
  "defaultVal": {
    "type": "none",
    "value": 5
  }
}
```



Please note that **"type"** must be set to **"none"**. The **"value"** property can be set in any format (string, number, ...).

Conditionally hide a field in a Multi-tenancy environment [🔗](#)

If you're configuring a MT environment (cm.multi_tenancy is set to true in the configuration), you can add a property `multiTenancyCheck: true` to a field (widget, field, column, etc).

This will tell the CM that the field must only be shown if:

1. The user has the landlord permission OR
2. The user is not a landlord, but he has more than 1 tenant

This verification does not replace the permissions verifications, it's only added to those verifications.

In a non Multi-tenancy environment, this condition is ignored.